



Level Order Successor (easy)

We'll cover the following ^

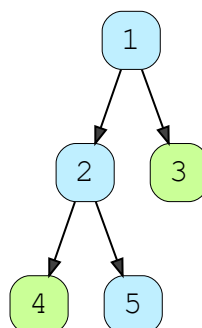
- Problem Statement
- Try it yourself
- Solution
- Code
- Time complexity
- Space complexity

Problem Statement

Given a binary tree and a node, find the level order successor of the given node in the tree. The level order successor is the node that appears right after the given node in the level order traversal.

Example 1:

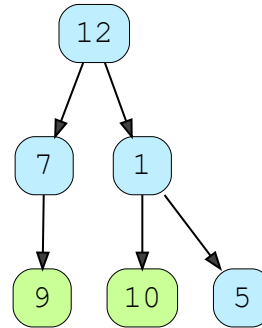
Given Node: 3
Level Order Successor: 4



Example 2:

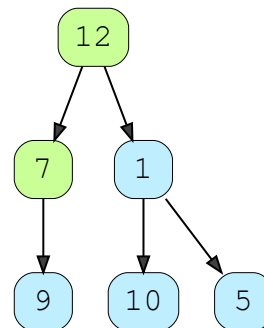


Given Node: 9
Level Order Successor: 10



Example 3:

Given Node: 12
Level Order Successor: 7



Try it yourself

Try solving this question here:

Java

Python3

JS

C++

```

1  import java.util.*;
2
3  class TreeNode {
4      int val;
5      TreeNode left;
6      TreeNode right;
7
8      TreeNode(int x) {
9          val = x;
10     }
11 }
12
13 class LevelOrderSuccessor {
14     public static TreeNode findSuccessor(TreeNode root, int key) {
15         // TODO: Write your code here
16         return null;
17     }
18
19     public static void main(String[] args) {
20         TreeNode root = new TreeNode(1);
21         root.left = new TreeNode(2);
22         root.right = new TreeNode(3);
23         root.left.left = new TreeNode(4);
24         root.left.right = new TreeNode(5);
25
26         TreeNode result = LevelOrderSuccessor.findSuccessor(root, 3);
  
```

```

26     TreeNode result = LevelOrderSuccessor.findSuccessor(root, 9);
27     if (result != null)
28         System.out.println(result.val + " ");
29
30     root = new TreeNode(12);
31     root.left = new TreeNode(7);
32     root.right = new TreeNode(1);
33     root.left.left = new TreeNode(9);
34     root.right.left = new TreeNode(10);
35     root.right.right = new TreeNode(5);
36
37     result = LevelOrderSuccessor.findSuccessor(root, 9);
38     if (result != null)
39         System.out.println(result.val + " ");
40
41     result = LevelOrderSuccessor.findSuccessor(root, 12);
42     if (result != null)
43         System.out.println(result.val + " ");
44 }
45 }
46

```

Run

Save

Reset



Solution

This problem follows the [Binary Tree Level Order Traversal](#) pattern. We can follow the same **BFS** approach. The only difference will be that we will not keep track of all the levels. Instead we will keep inserting child nodes to the queue. As soon as we find the given node, we will return the next node from the queue as the level order successor.

Code

Here is what our algorithm will look like; most of the changes are in the highlighted lines:

Java

Python3

C++

JS

```

1  import java.util.*;
2
3  class TreeNode {
4      int val;
5      TreeNode left;
6      TreeNode right;
7
8      TreeNode(int x) {
9          val = x;
10     }
11 };
12
13 class LevelOrderSuccessor {
14     public static TreeNode findSuccessor(TreeNode root, int key) {
15         if (root == null)
16             return null;
17
18         Queue<TreeNode> queue = new LinkedList<>();
19         queue.offer(root);
20         while (!queue.isEmpty()) {
21             TreeNode currentNode = queue.poll();
22             // insert the children of current node in the queue
23             if (currentNode.left != null)
24                 queue.offer(currentNode.left);
25             if (currentNode.right != null)
26                 queue.offer(currentNode.right);
27

```



```
27
28     // break if we have found the key
29     if (currentNode.val == key)
30         break;
31 }
32
33 return queue.peek();
34 }
35
36 public static void main(String[] args) {
37     TreeNode root = new TreeNode(1);
38     root.left = new TreeNode(2);
39     root.right = new TreeNode(3);
40     root.left.left = new TreeNode(4);
41     root.left.right = new TreeNode(5);
42
43     TreeNode result = LevelOrderSuccessor.findSuccessor(root, 3);
44     if (result != null)
45         System.out.println(result.val + " ");
46
47     root = new TreeNode(12);
48     root.left = new TreeNode(7);
49     root.right = new TreeNode(1);
50     root.left.left = new TreeNode(9);
51     root.right.left = new TreeNode(10);
52     root.right.right = new TreeNode(5);
53
54     result = LevelOrderSuccessor.findSuccessor(root, 9);
55     if (result != null)
56         System.out.println(result.val + " ");
57
58     result = LevelOrderSuccessor.findSuccessor(root, 12);
59     if (result != null)
60         System.out.println(result.val + " ");
61 }
62 }
63
```

Run

Save

Reset



Time complexity

The time complexity of the above algorithm is $O(N)$, where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space complexity

The space complexity of the above algorithm will be $O(N)$ which is required for the queue. Since we can have a maximum of $N/2$ nodes at any level (this could happen only at the lowest level), therefore we will need $O(N)$ space to store them in the queue.

